Neural Recipe Selection for Logic Synthesis Quality and Compute on Unseen Circuits

Shaan Chopra (240905140), Kenneth Rodrigues (240905112),
Darren Pais (240905093), and Aniket Verma (240905156)

School of Computer Science and Engineering

Manipal Institute of Technology

October 2026

Abstract

Trying many logic-synthesis recipes on every new circuit is expensive. We train a small neural network and classical supervised models to recommend recipes from a fixed library of 120 sequences. Our dataset contains 24,600 measured outcomes from 205 public circuits. On unseen circuits, one neural recommendation gives 0.19% mean improvement over the best fixed training recipe, with a 95% interval of [-0.40%, 0.79%]. Extra-Trees gives a larger 1.11% mean gain. Executing the NN's top five recommendations gives 5.12% improvement and a 3.99% mean gap to exhaustive search. In a separate 12-circuit serial timing audit, one and five neural recommendations are 32.2 and 15.4 times faster than evaluating all 120 recipes, including feature collection. Global shortlists achieve similar quality with less overhead. The study measures how much synthesis quality a small execution budget retains.

1 Problem and Approach

A digital circuit computes a function of binary inputs. Different networks of logic gates can implement the same function with different amounts of chip area and different output delays. *Logic synthesis* transforms the network while preserving its function. A sequence of transformations is a *recipe*; its order matters because each transformation changes the opportunities available to the next.

A standard recipe gives a quick general-purpose choice. Exhaustive search runs every candidate separately on a new circuit and keeps the best measured result. We study whether a model can use experience from earlier circuits to recommend a useful recipe after only one or a few complete synthesis runs.

We first measure all 120 recipes on each training circuit. This is an offline dataset-building cost. At deployment, a model receives features of a new circuit, ranks the known recipes, and recommends one or a short list. We execute those recommendations and retain the best measured result. Figure 1 separates the two stages.

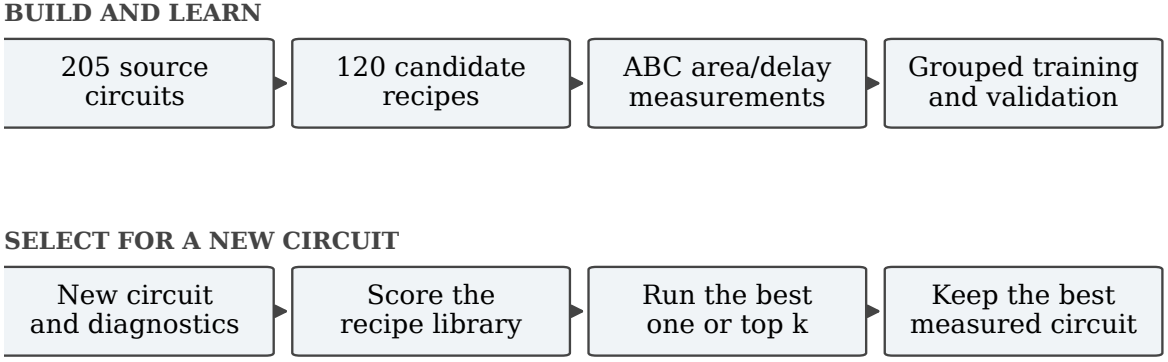


Figure 1: Exhaustive measurements supply training labels. A new test circuit contributes features only; its recommended recipes are then executed.

Our main references are the *best fixed recipe*, chosen by its average training performance, and the *oracle*, the best of all 120 measured recipes for each circuit. The aim is to retain useful quality while reducing repeated search cost. We also compare neural recommendations with tree models, linear models, and globally ranked shortlists.

2 Circuits, Recipes, and Data

2.1 What the synthesis tool measures

We use combinational circuits: their outputs depend on their current inputs, with no registers or memory. ABC [1] represents them as an *And-Inverter Graph* (AIG), whose nodes compute two-input AND and whose edges may invert a signal. Shared nodes represent reused computation. Node count describes graph size; *depth* is the longest chain of AND nodes from an input to an output.

ABC's `strash` command constructs this graph and merges identical AND nodes. Recipes then rewrite, refactor, reuse, or balance its logic (Table 1). The standard `resyn2` recipe is the ten-operation sequence `b rw rf b rw rwz b rfz rwz b` [2]. It appears in our library as R001.

Operation	Effect on the current graph
`rw`, `rwz`	Rewrite small logic regions using equivalent structures.
`rf`, `rfz`	Refactor larger regions, for example $ab + ac = a(b + c)$.
`rs`, `rsz`	Resubstitute a function using signals already available in the circuit.
`b`	Balance chains of logic to reduce depth.

Table 1: The seven recipe operations. A `z` variant permits a replacement with no immediate node-count reduction, which can help a later operation.

After applying a complete recipe, *technology mapping* assigns the graph to standard cells: predesigned gates with known functions, areas, and timing models. We use the NanGate 45 nm library at its typical operating condition [5]. *Area* is the sum of cell areas in μm^2; *delay* is the estimated slowest input-to-output path in picoseconds. Smaller values are preferable, although improving one can worsen the other. These are synthesis estimates before placement

and routing.

Each candidate starts in a fresh ABC process from the same original circuit. We read the cell library and circuit, run `strash`, apply the recipe in order, and run `map; topo; stime` to obtain area and delay. Candidates never inherit another candidate's optimized graph. A separate run omits the recipe and supplies the circuit's baseline area A_0 and delay D_0.

2.2 Dataset and objective

We use 205 public circuits: EPFL arithmetic and control designs [3], plus ISCAS-85 and MCNC circuits from the ISP RAS benchmark collection [4]. Table 2 shows their sizes. The pilot inventory retains combinational designs with at least four inputs and 40 AND nodes, subject to 30,000-node EPFL and 8,000-node MCNC limits. It removes overlapping ISCAS-85 copies and three duplicate signatures based on input, output, node, and depth counts. These counts provide coarse screening; source-file checksums identify the retained files.

Family	**Circuits**	**Min. nodes**	**Median**	**Max. nodes**	**Depth median**
EPFL arithmetic	7	1,020	5,416	27,062	255
EPFL control	9	174	693	13,758	20
ISCAS-85	10	209	610	2,337	30
MCNC	179	40	270	7,803	14

Table 2: Circuit inventory. Size and depth describe the original AIG.

The library preserves 60 pilot recipes and adds 60 with random seed 17. Each contains 6–12 operations. It includes `resyn` and `resyn2`, 24 area-weighted, 24 delay-weighted, 24 mixed, and 46 uniformly sampled sequences. These groups describe operation-sampling preferences; their actual quality comes from measurement. Duplicate sequences and consecutive balancing operations are rejected.

All $205 \times 120 = 24,600$ experiments completed successfully. Each row records the circuit features, recipe, mapped area and delay, baseline, and runtime. We normalize by the circuit's own baseline and use equal area and delay weights:

$$y_A = \frac{A}{A_0}, \qquad y_D = \frac{D}{D_0}, \qquad J = \tfrac{1}{2}y_A + \tfrac{1}{2}y_D. \tag{1}$$

Lower J is better. For example, $y_A = 0.90$ means a 10% area reduction from the unoptimized baseline. Normalization lets circuits of different sizes contribute comparably. We also test area-only and delay-only objectives.

Recipe	**Area (μm^2)**	**Delay (ps)**	y_A	y_D	$J_{0.5}$
R001 (resyn2)	385.97	621.87	1.083	0.929	1.006
R037	343.94	602.63	0.965	0.900	0.932

Table 3: Measured example: circuit `c1908`, with baseline area $356.44\,\mu$m^2 and delay 669.63 ps. R037 improves both quantities relative to `resyn2`.

3 Learning to Select a Recipe

3.1 Features available before recipe execution

Our full circuit representation has 50 features:

Graph statistics (24) describe inputs, outputs, node count, depth, fanout, and the distribution of nodes across levels. Fanout counts how many connections use a signal. Other statistics describe unequal input-path lengths, inverted edges, exclusive-OR patterns, and paths that split and rejoin. We apply $\log(1+x)$ to ten size-like features.

Mapping features (4) are $\log A_0$, $\log D_0$, baseline area per AND node, and baseline delay per graph level.

Probe responses (22) measure how the graph reacts to short diagnostic transformations. Seven single-operation probes and four two-operation probes (`b rw`, `rw b`, `rf b`, `b rf`) each provide node-count and depth ratios relative to the original graph. Every probe restarts from that graph and stops before mapping. Probes reveal useful behavior, but take time to collect. A second NN uses only the 24 graph statistics and skips them.

Classical regressors also receive a 144-feature recipe encoding: length and `z`-count (2), operation counts (7), identities at 12 positions (84), adjacent-operation counts (49), and first/last balancing indicators (2). This distinguishes sequences with the same operation counts but different order. The full circuit–recipe input has 194 features. Circuit identity, source family, post-recipe measurements, and runtime are excluded.

3.2 Neural network and other models

The selector learns differences between recipes on the same circuit. We center each training circuit's objective across its 120 candidates, giving targets $\widetilde{J}(C,R) = J(C,R) - \mathrm{mean}_{R'}\, J(C,R')$. Removing this shared offset preserves the recipe ranking.

Our multilayer perceptron has 50 inputs, hidden layers of 64 and 32 ReLU units, and 120 linear outputs: 9,304 parameters. One circuit is one training example with 120 measured targets. Each output scores a known recipe, so a single forward pass ranks the whole library. The 24-input variant has the same hidden and output layers.

For recipe R, let μ_R be its mean centered training outcome and s the pooled standard deviation of training residuals. The NN predicts

$$\widehat{\widetilde{J}}(C,R) = \mu_R + s f_\theta(x_C)_R. \tag{2}$$

It learns $(\widetilde{J} - \mu_R)/s$ using mean squared error. We standardize inputs using training-circuit statistics and initialize the final layer to zero, so training begins with the global mean recipe ranking. AdamW uses learning rate 0.003, weight decay 0.03, full batches, and hidden-layer dropout 0.1. Inner validation chooses 25, 75, or 200 epochs. The fixed 120-output design requires refitting when the recipe library changes.

We also train additive ridge regression, interaction ridge, Extra-Trees, Random Forest, and LightGBM. Interaction ridge adds 154 probe-by-operation-count products. Additive ridge gives every circuit the same recipe ordering; interactions and nonlinear models allow circuit-specific rankings. Table 4 records their small tuning grids.

At deployment, each model selects its lowest-scored recipe. A top-k selector instead executes its k highest-ranked candidates and keeps the best measured J, with $k \in \{1,2,3,5,8\}$. The

Classical model	Settings
Ridge variants	Training-fitted standardization; $\alpha \in \{10, 100\}$.
Extra-Trees, Random Forest	80 trees; depth at most 16; 30% feature sampling; minimum leaf size in $\{5, 15\}$.
LightGBM	180 trees; learning rate 0.05; minimum leaf size 20; L_2 penalty 1; leaves in $\{15, 31\}$.

Table 4: Hyperparameters are selected on inner training folds. NN settings are described above.

adaptive classical selector chooses its model family through inner validation. A margin-gated variant falls back to the fixed recipe unless its predicted advantage exceeds a tuned threshold in $\{0, 0.0025, 0.005, 0.01, 0.02\}$.

4 Evaluation

4.1 Unseen circuits and reference methods

The independent examples are the 205 circuits. All 120 recipe rows for a circuit stay in the same partition. We use five outer folds repeated with seeds 0, 1, and 2, stratified by source family where possible: approximately 164 training and 41 test circuits per fold. Every circuit is tested once per repeat.

Three-fold inner validation selects hyperparameters by mean percentage gain over the best fixed inner-training recipe. All feature scaling, target statistics, and family selection use the relevant training partition. NN and classical models use identical outer splits. A test decision is made from features and predictions before looking up the selected recipe's outcome. Only the chosen k outcomes are available to a top-k selector; the complete outcome table supplies evaluation references.

The **best fixed recipe** minimizes mean training J and is reused on every test circuit. A **global top-k shortlist** executes the k recipes with the lowest training means and keeps their best result. We also report the standard `resyn2` script and the exact expected outcome of uniformly sampling k distinct recipes. The **oracle** keeps the best of all 120 measured candidates on each test circuit.

For each circuit, we measure gain over the fixed recipe and the remaining gap to the oracle:

$$\text{Gain} = 100\frac{J_{\text{fixed}} - J_{\text{selected}}}{J_{\text{fixed}}}, \qquad \text{Gap} = 100\frac{J_{\text{selected}} - J_{\text{oracle}}}{J_{\text{oracle}}}. \tag{3}$$

Positive gain and smaller gap are better. We average the three held-out results within each circuit, then bootstrap the 205 circuit averages with 2,000 resamples for 95% intervals. An interval covering zero leaves the direction of a mean gain uncertain; it does not establish statistical equivalence.

4.2 Timing and additional checks

We separately rerun all 120 recipes on 12 circuits chosen near the 25th, 50th, and 75th AIG-size percentiles of each source family. Each recommendation uses the first repeat's saved model that held that circuit out. All 1,440 fresh area and delay results reproduce the dataset.

Serial cost includes graph extraction, baseline mapping, any probes, prediction, and all selected

recipe runs. Even the graph-only NN includes baseline mapping to report the normalized objective. Global shortlists with $k > 1$ need that mapping to compare candidates; a single fixed recipe can run directly. Exhaustive cost is baseline mapping plus all 120 candidate times. Prediction includes preprocessing, takes the median of seven runs, and uses an already loaded model. Model loading and application startup are separate costs.

Additional experiments examine family transfer, feature and target ablations, recipe encodings, smaller training sets and libraries, and recipes withheld from training. Controlled ablations use boosting and one five-fold repeat; family transfer uses the adaptive classical selector. Appendix A retains their numerical results. The unseen-recipe experiment trains on 96 recipes and offers all 120 at test time; this tests the classical sequence encoding, while the NN retains its fixed output library.

5 Results and Compute Efficiency

5.1 Quality retained with fewer runs

The dataset sweep took 30.7 minutes with five ABC workers. The median circuit has 327 AND nodes and depth 16. The oracle chooses 85 different recipes across the pool, showing that useful choices vary by circuit. Table 5 compares one-recipe selection; Table 6 shows larger budgets.

One-recipe selector	**Gain (%)**	**95% interval**	**Gap (%)**
NN: 50 circuit features	0.19	[-0.40, 0.79]	9.60
NN: 24 graph features	0.44	[-0.28, 1.10]	9.25
Additive ridge	0.12	[-0.06, 0.31]	9.81
Interaction ridge	0.02	[-0.75, 0.78]	9.74
Extra-Trees	1.11	[0.60, 1.62]	8.55
Random Forest	0.68	[0.04, 1.28]	9.06
Gradient boosting	0.45	[-0.18, 1.02]	9.29
Adaptive classical selector	0.73	[0.25, 1.18]	9.01
Adaptive + margin gate	0.72	[0.24, 1.16]	9.04
Uniform random (expected)	-1.94	[-2.53, -1.32]	11.83
resyn2	-0.02	[-0.94, 0.86]	9.66
Best fixed training recipe	0.00	[0.00, 0.00]	9.97
Exhaustive-search oracle	8.61	[7.90, 9.39]	0.00

Table 5: Quality over all 205 circuits and three validation repeats. Each selector executes one recipe; the oracle evaluates 120. Gain is relative to the best fixed training recipe. Gap is the remaining percentage above the oracle.

One full-NN recommendation gave 0.19% mean gain over the best fixed recipe (95% interval [-0.40%, 0.79%]); the graph-only NN gave 0.44% ([-0.28%, 1.10%]). Their average quality is close to the fixed recipe, with uncertainty about whether either improves it. Extra-Trees gave the strongest one-recipe mean gain, 1.11% ([0.60%, 1.62%]), while the inner-selected classical family gave 0.73%.

Executing the full NN's top five recommendations improved the fixed recipe by 5.12% (95% interval [4.54%, 5.70%]) and reduced mean oracle gap to 3.99%. It found an optimum among the 120 recipes on 15.28% of held-out predictions and came within 5% on 70.24%. The global

top-five baseline reached 4.13% gap. The NN's mean gain over that shortlist was 0.06% ([-0.36%, 0.48%]).

The full NN's one-recipe choices reduced mean area by 0.31% and increased mean delay by 0.47% relative to the fixed recipe. With five recommendations, mean area and delay fell by 2.44% and 7.17%. The combined objective permits these trade-offs.

5.2 Time saved on a new circuit

Selector	**Runs**	**Gap (%)**	**Time (s)**	**Speedup (×)**
Best fixed recipe	1	9.97	0.41	110.4
NN: 50 features	1	9.60	1.39	32.2
NN: 24 features	1	9.25	0.77	58.0
Global top five	5	4.13	2.22	20.2
NN: top five	5	3.99	2.92	15.4
Graph-only NN: top five	5	4.35	2.31	19.4
Global top eight	8	3.00	3.41	13.1
Exhaustive search	120	0.00	44.84	1.0

Table 6: Quality and execution budget. Gap uses all 205 circuits; time uses the separate 12-circuit serial audit. Time includes features and executed recipes with the model loaded. Speedup divides mean exhaustive time by mean selector time on those same 12 circuits.

Exhaustive search averaged 44.84 seconds per timing circuit. One full-NN recommendation took 1.39 seconds, including features and synthesis: 32.2 times faster, saving 96.89% of elapsed time. Its top five took 2.92 seconds, a 15.4-fold speedup and 93.49% saving. The graph-only NN used 0.77 seconds for one recipe.

Global top-five search took 2.22 seconds and achieved similar quality. Most savings therefore come from executing fewer recipes. Neural selection adds circuit-specific ranking and feature-collection cost. The cheapest option remains the best fixed recipe. These speedups describe serial search with a loaded predictor; parallel search and a fresh model launch have different elapsed costs.

Building the training table took 30.7 minutes with five ABC workers. NN evaluation and inner tuning took 90.7 seconds; an outer-fold NN refit averaged 0.10 seconds after selecting its settings. The table is a shared offline cost for the fixed baseline and learned models. A new circuit needs only feature extraction, prediction, and the selected recipe runs.

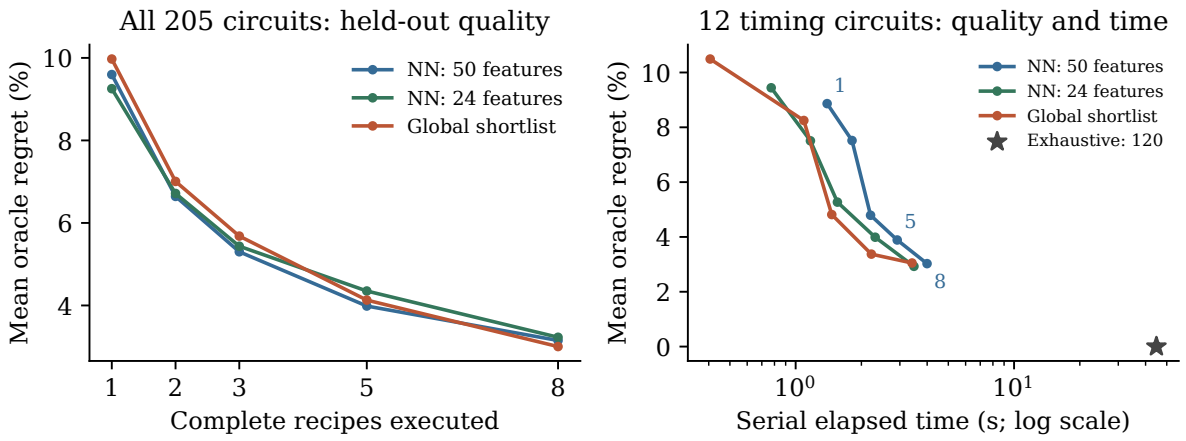


Figure 2: Left: held-out quality over 205 circuits as more recipes are executed. Right: fresh serial time and quality on the 12 timing circuits; blue labels mark the full NN's recipe budget. The exhaustive reference evaluates all 120 candidates.

5.3 What the additional experiments show

In the controlled boosting repeat, all features gave 0.81% gain, static graph features alone gave 0.00%, and probe features alone gave 0.51%. Rank-based targets gave 1.13%. Training the adaptive classical selector on MCNC and testing on the other 26 circuits gave 1.61% gain, with interval [-0.06%, 3.31%]. Transfer to the nine held-out control circuits gave -2.69%, illustrating sensitivity to circuit type.

Area-only and delay-only gains, smaller training/library experiments, and the 24-withheld-recipe result all had intervals covering zero. The withheld-recipe point estimate was 0.66%. These results support testing on the intended circuit population and retaining simple shortlist baselines. The pool has 179 MCNC circuits, so its overall average mainly reflects that family.

6 Discussion and Practical Use

The NN produces one-recipe quality near the best fixed training recipe while avoiding a new 120-candidate sweep. Its top five leave a 3.99% mean gap to the library optimum and save 93.49% of serial search time in the timing audit. Extra-Trees gives a stronger one-recipe mean gain, and global shortlists offer similar shortlist quality with less overhead. The useful choice therefore depends on the allowed execution budget and feature cost.

The conclusions concern 205 benchmark-scale combinational circuits, one cell library, and synthesis timing before placement and routing. Larger or sequential circuits and other technologies need additional measurements. The training table remains an upfront cost shared by the learned selectors and fixed-recipe baseline.

Our recipe operations follow ABC's AIG rewriting work [2]. DRiLLS constructs synthesis sequences through reinforcement learning [6]; FlowTune explores flows with a multi-armed bandit [7]. OpenABC-D provides labeled synthesis data for supervised learning [8]. We focus on selecting from a finite library and measuring the quality and execution cost of the selected recipes.

The release includes code, data, fold predictions, trained models, and source checksums. All 12,300 overlapping pilot measurements reproduced, and 24 sampled mapped circuits passed

Boolean-equivalence checks. We also reproduced all 147,600 saved NN scores. Tool and source commits are pinned in the manifest; package versions are recorded in `requirements.lock.txt`. Experiments use Python 3.12, PyTorch 2.11, scikit-learn 1.8, and LightGBM 4.6 on eight logical CPUs; the timing audit uses one synthesis worker.

The supplied dataset supports direct reproduction through the commands in `README.md`. To optimize a new BLIF, BENCH, or AIGER circuit with five neural recommendations and write the best mapped netlist:

```
circuit-recipe select design.blif --verify 5 --netlist out.blif
```

The command reports measured area, delay, objective, and timing. The default NN is refitted on all 205 circuits for deployment; reported quality comes from held-out predictions. The graph-only NN and classical artifact can be selected with `-model`.

References

[1] Berkeley Logic Synthesis and Verification Group. ABC: A System for Sequential Synthesis and Verification. `https://github.com/berkeley-abc/abc`.

[2] A. Mishchenko, S. Chatterjee, and R. Brayton. DAG-Aware AIG Rewriting: A Fresh Look at Combinational Logic Synthesis. In *DAC*, pp. 532–535, 2006. `https://doi.org/10.1145/1146909.1147048`.

[3] L. Amarù, P.-E. Gaillardon, and G. De Micheli. The EPFL Combinational Benchmark Suite. In *IWLS*, 2015. Benchmark repository: `https://github.com/lsils/benchmarks`.

[4] ISP RAS. hdl-benchmarks: Collection of Digital Hardware Modules and Projects. `https://github.com/ispras/hdl-benchmarks`.

[5] The OpenROAD Project. NanGate45 Platform and Typical-Corner Open Cell Library. `https://github.com/The-OpenROAD-Project/OpenROAD-flow-scripts/tree/master/flow/platforms/nangate45`.

[6] A. Hosny, S. Hashemi, M. Shalan, and S. Reda. DRiLLS: Deep Reinforcement Learning for Logic Synthesis. In *ASP-DAC*, pp. 581–586, 2020. `https://arxiv.org/abs/1911.04021`.

[7] W. Lau Neto, Y. Li, P.-E. Gaillardon, and C. Yu. FlowTune: End-to-End Automatic Logic Optimization Exploration via Domain-Specific Multi-Armed Bandit. *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, 2022. `https://arxiv.org/abs/2202.07721`.

[8] A. B. Chowdhury, B. Tan, R. Karri, and S. Garg. OpenABC-D: A Large-Scale Dataset for Machine Learning Guided Integrated Circuit Synthesis. arXiv:2110.11292, 2021. `https://arxiv.org/abs/2110.11292`.

A Additional Experiment Results

Table 7 preserves the feature, target, encoding, family-transfer, and sensitivity comparisons. All use one executed recipe. Boosting controls use one five-fold repeat with inner tuning. Transfer tests exclude the stated family from training and tuning; the MCNC-to-other test contains 26 circuits. Objective and library changes each use their own training-only fixed comparator.

Setting	Gain (%)	95% interval	Gap (%)
Ablations: boosting, first five-fold repeat			
All features; centered target	0.81	[-0.02, 1.62]	8.86
Static graph features	0.00	[-1.11, 1.02]	9.76
Static + mapping	0.21	[-0.89, 1.30]	9.47
Probe features alone	0.51	[-0.39, 1.44]	9.21
Raw objective target	0.77	[-0.07, 1.61]	8.87
Within-circuit rank target	1.13	[0.20, 2.02]	8.42
Operation counts	0.55	[-0.45, 1.48]	9.10
Counts + positions	0.29	[-0.86, 1.36]	9.42
Family transfer: adaptive classical selector			
MCNC to other families (26)	1.61	[-0.06, 3.31]	8.75
EPFL arithmetic held out (7)	0.95	[-2.49, 4.91]	8.72
EPFL control held out (9)	-2.69	[-9.61, 3.16]	15.77
ISCAS-85 held out (10)	0.98	[-0.35, 2.38]	8.72
MCNC held out (179)	0.00	[0.00, 0.00]	10.34
Sensitivity: boosting, first five-fold repeat			
Delay-only objective	-0.79	[-2.27, 0.70]	18.61
Area-only objective	-0.28	[-1.05, 0.48]	7.41
30-recipe library	0.36	[-0.45, 1.15]	6.91
60-recipe library	0.37	[-0.59, 1.30]	8.05
25 training circuits	0.19	[-0.65, 1.09]	9.30
50 training circuits	0.13	[-0.73, 0.95]	9.41
100 training circuits	-0.62	[-1.69, 0.37]	9.69
24 recipes withheld from training	0.66	[-0.28, 1.56]	8.95

Table 7: Additional checks. Gain is relative to the fixed training recipe; gap is relative to the available library oracle. Numbers in transfer labels give test-circuit counts.

B Feature and Run Details

For input count I, output count O, AND count N, and depth L, the 24 graph statistics comprise: those four counts; edge count $2N+O$; ratios N/L, $(2N+O)/(N+I)$, I/N, and O/N; five fanout statistics (mean, maximum, standard deviation, fraction above one, and mean over AND nodes); four level-width statistics (maximum, mean, coefficient of variation, and maximum divided by N); three input-level imbalance statistics (mean, maximum, fraction at least two); and fractions of inverted edges, local XOR patterns, and local reconvergence. Reconvergence checks for a shared signal within one predecessor step. Ten size-related columns receive $\log(1 + x)$; ratios retain their scale.

The complete operations and explicit ABC commands are in `data/processed/recipes.csv`; circuit paths and hashes are in `circuits.csv`. ABC is pinned to commit `5d64f3b5`, with full source and cell-library hashes in `manifest.json`. Outer seeds are 0, 1, and 2; inner seeds are the fit seed plus 1000; the bootstrap uses seed 2026. Saved split files and tuning records reproduce every partition and model choice. The earlier detailed paper and its source are retained alongside this concise version.